University of Asia Pacific
Department of Computer Science and Engineering

Compiler Design
(CSE 430)

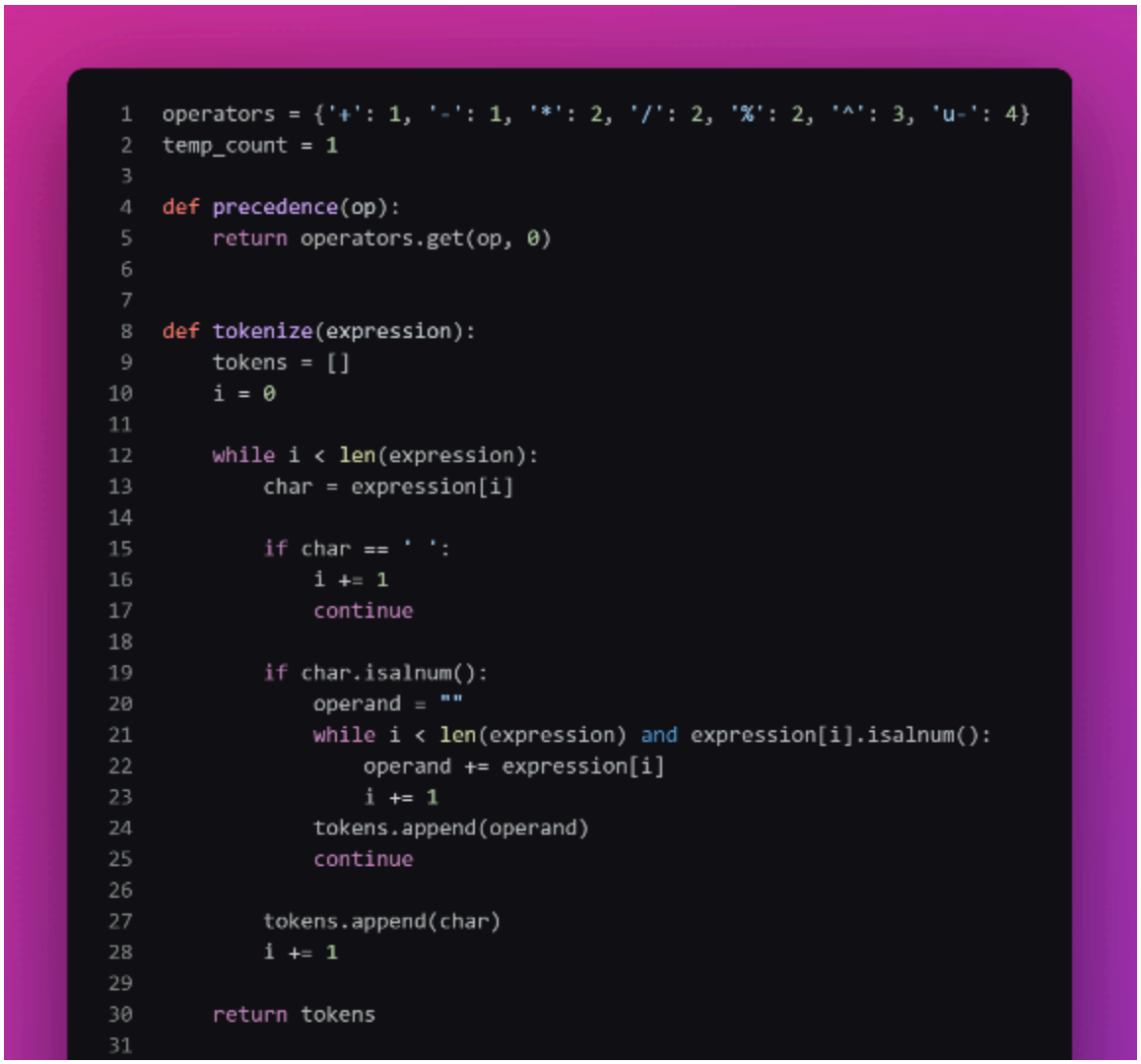
Submitted By
Mazharul Islam Sourav
ID: 22101133
Section: C-2
Semester: 4-2

Submitted To
Md Shafayatul Haque
Lecturer
Department of Computer Science and Engineering
University of Asia Pacific

Work 05: Three Address Code Generation.

Code -

Tokenization Function -



```python
operators = {'+': 1, '-': 1, '*': 2, '/': 2, '%': 2, '^': 3, 'u-': 4}
temp_count = 1

def precedence(op):
    return operators.get(op, 0)


def tokenize(expression):
    tokens = []
    i = 0

    while i < len(expression):
        char = expression[i]

        if char == ' ':
            i += 1
            continue

        if char.isalnum():
            operand = ""
            while i < len(expression) and expression[i].isalnum():
                operand += expression[i]
                i += 1
            tokens.append(operand)
            continue

        tokens.append(char)
        i += 1

    return tokens

```

PostFix Function -

```python

def postfix(tokens):
    stack = []
    postfix = []

    for i, token in enumerate(tokens):

        if token not in '+-*/%^()':
            postfix.append(token)

        elif token == '(':
            stack.append(token)

        elif token == ')':
            while stack and stack[-1] != '(':
                postfix.append(stack.pop())
            stack.pop()

        elif token == '-' and (i == 0 or tokens[i-1] in '(+-*/%^'):
            stack.append('u-')

        else:
            while stack and stack[-1] != '(':
                if token == '^' or stack[-1] == 'u-':
                    if precedence(stack[-1]) > precedence(token):
                        postfix.append(stack.pop())
                    else:
                        break
                else:
                    if precedence(stack[-1]) >= precedence(token):
                        postfix.append(stack.pop())
                    else:
                        break

            stack.append(token)

    while stack:
        postfix.append(stack.pop())

    return postfix

```

Intermediate Code Generation Function -

```python

def TAC(postfix):
    global temp_count

    stack = []
    tac = []

    for token in postfix:

        if token == 'u-':
            operand = stack.pop()
            temp = f"T{temp_count}"
            temp_count += 1
            tac.append(f"{temp} = uminus {operand}")
            stack.append(temp)

        elif token in operators:
            right = stack.pop()
            left = stack.pop()
            temp = f"T{temp_count}"
            temp_count += 1
            tac.append(f"{temp} = {left} {token} {right}")
            stack.append(temp)

        else:
            stack.append(token)

    return tac

```

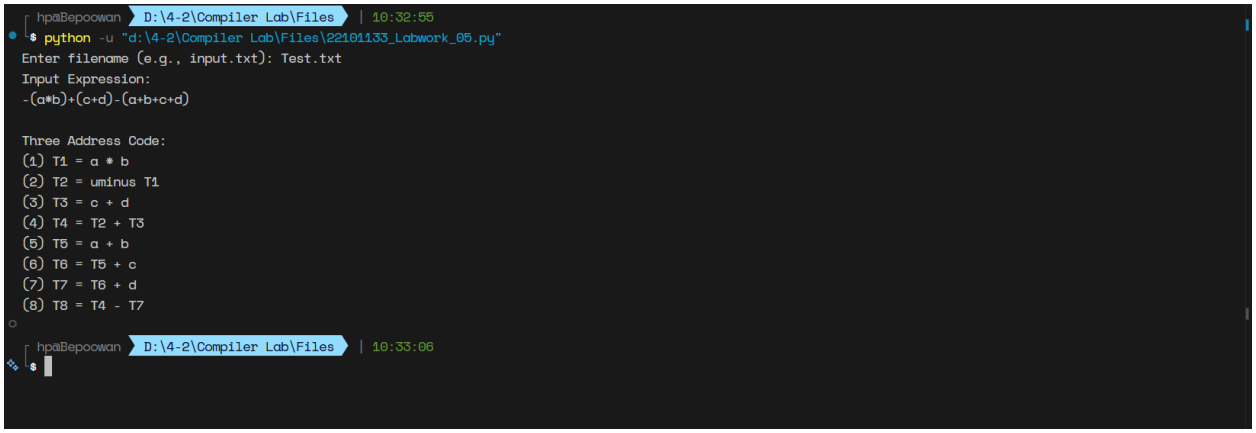
Main Function -

```
filename = input("Enter filename (e.g., input.txt): ")

try:
    with open(filename, 'r') as file:
        expression = file.read().strip()

    print("Input Expression:")
    print(expression)
    print()

    tokens = tokenize(expression)
    postfix = postfix(tokens)
    tac_code = TAC(postfix)

    print("Three Address Code:")
    for i, line in enumerate(tac_code, 1):
        print(f"({i}) {line}")

except FileNotFoundError:
    print(f"Error: File '{filename}' not found!")
except Exception as e:
    print(f"Error: {e}")
```

Input -

```
1   -(a*b)+(c+d)-(a+b+c+d)
```

Output -



Code Link : [GitHub]